

ISIT307 - WEB SERVER PROGRAMMING

LECTURE 6.1 – MANAGING STATE INFORMATION



LECTURE PLAN

- Learn about state information
- Use hidden form fields to save state information
- Use query strings to save state information
- Use cookies to save state information
- Use sessions to save state information

STATE INFORMATION

- Information about individual visits to a Web site is called **state information**
- HTTP was originally designed to be **stateless** – Web browsers store no persistent data about a visit to a Web site
- **Maintaining state** means to store persistent information about Web site visits

STATE INFORMATION

- Customize individual Web pages based on user preferences
- Allow a user to create bookmarks for returning to specific locations within a Web site
- Provide shopping carts that store order information
- Store user IDs and passwords
- Keep track of users' navigation

STATE INFORMATION

- The four tools for maintaining state information with PHP are:
 - Hidden form fields
 - Query strings
 - Cookies
 - Sessions

USING HIDDEN FORM FIELDS TO SAVE STATE INFORMATION

- **Hidden form fields can be used to** temporarily store data that needs to be sent to a server (without be shown to the user)
- Hidden form fields are created with the `<input>` element

`<input type="hidden">`

USING HIDDEN FORM FIELDS TO SAVE STATE INFORMATION

- When submitting a form to a PHP script, access to the values submitted from the form is with the `$_GET[]` and `$_POST[]` autoglobals
- The disadvantages of hidden boxes
 - Increased transmission times as the state data are sent back and forth between the client and server for each request
 - Requiring requests sent to the server are preferred to be “post” instead of “get”
 - Potential for being altered by the user maliciously

USING HIDDEN FORM FIELDS TO SAVE STATE INFORMATION - EXAMPLE

```
echo "<form method='post' " . " action='AvailableOpportunities.php'>\n";  
  
. . .  
  
echo "<input type='hidden' name='internID' " . " value='$InternID'>\n";  
echo "<input type='submit' name='submit' " .  
    " value='View Available Opportunities'>\n";  
  
echo "</form>\n";
```

USING QUERY STRINGS TO SAVE STATE INFORMATION

- A **query string** is a set of name=value pairs appended to a target URL
- Add a question mark (?) immediately after the URL followed by the query string that contains the information to be preserved in name/value pairs

USING QUERY STRINGS TO SAVE STATE INFORMATION - EXAMPLE

- Example

```
<a href="http://www.example.com/TargetPage.php?  
        firstName=Elena&lastName=Vlahu&  
        occupation=lecturer">Link Text</a>
```

- Accessing query string data

```
echo "{$_GET['firstName']} {$_GET['lastName']} is a  
{$_GET['occupation']}. ";
```

USING COOKIES TO SAVE STATE INFORMATION

- Query strings do not permanently maintain state information
- To store state information beyond the current Web page session, cookies can be created
- **Cookies**, or magic cookies, are small pieces of information about a user/session that are stored by a Web server in text files on the user's computer

USING COOKIES TO SAVE STATE INFORMATION

- **Temporary cookies** remain available only for the current browser session
- **Persistent cookies** remain available beyond the current browser session and are stored in a text file on a client computer

CREATING COOKIES

- `setcookie()` - function that creates and modify cookies

```
setcookie(name [,value ,expires, path, domain,  
secure, httponly, samesite])
```

- "name" - name of the cookie
- "value" - value assigned
- "expires" - cookie expiration
- "path" - the path for which the cookie will be available
- "domain" - the domain where the cookie will be available
- "secure" - when set to true, the cookie will only be transmitted if the request is made via HTTPS
- "httponly" - when set to true, the cookie cannot be accessed through scripts (such as JavaScript)
- "samesite" - when set to strict, the cookie will only be sent on requests originating from the same registered website

CREATING COOKIES

- `setcookie()` should be called before sending the Web browser any output, including white space, HTML elements, or output from the `echo()` or `print()` statements
- Users can choose whether to accept cookies that a script attempts to write to their system
- A value of `TRUE` is returned even if a user rejects the cookie

```
setcookie("name", "Elena", time() + 3600, '/',  
'domain.com', true, true, 'Strict');
```

CREATING COOKIES

- By default, cookies cannot include semicolons or other special characters, such as commas or spaces, that are transmitted between Web browsers and Web servers using HTTP
- Cookies **can** include special characters when created with PHP since encoding converts special characters in a text string to their corresponding hexadecimal ASCII value

CREATING COOKIES

- `setcookie()` can be called multiple times to create additional or update (modify) cookies – as long as the `setcookie()` statements come before any other output on a Web page

```
setcookie("firstName", "Elena");
```

```
setcookie("lastName", "Vlahu");
```

```
setcookie("occupation", "lecturer");
```

```
setcookie("firstName", "Elena", time()+3600);
```

READING COOKIES

- Cookies that are available to the current Web page are automatically assigned to the `$_COOKIE` autoglobal
- Each cookie can be accessed by using the cookie name as a key in the associative `$_COOKIE[]` array

```
echo $_COOKIE['firstName'];
```

- Newly created cookies are not available until after the current Web page is reloaded

READING COOKIES

- To ensure that a cookie is set before using it, the `isset()` function can be used

```
setcookie("firstName", "Elena");
setcookie("lastName", "Vlahu");
setcookie("occupation", "lecturer");
---
if (isset($_COOKIE['firstName'])
    && isset($_COOKIE['lastName'])
    && isset($_COOKIE['occupation']))
    echo "{$_COOKIE['firstName']} {$_COOKIE['lastName']}
        is a {$_COOKIE['occupation']}.";
```

DELETING COOKIES

- To delete a persistent cookie before the time assigned to the `expires` argument elapses, a new expiration value that is sometime in the past should be assigned
- This can be done by subtracting any number of seconds from the `time()` function

```
setcookie("firstName", "", time()-3600);  
setcookie("lastName", "", time()-3600);  
setcookie("occupation", "", time()-3600);
```

USING SESSIONS TO SAVE STATE INFORMATION

- A **session** refers to a period of activity when a PHP script stores state information on a Web server
- Sessions allow maintaining state information even when clients disable cookies in their Web browsers

STARTING A SESSION

- `session_start()` - function that starts a new session or continues an existing one
 - does not accept any arguments, nor return a value
 - generates a unique session ID (random alphanumeric string) to identify the session
 - creates a text file on the Web server that is the same name as the session ID, preceded by `sess_`

STARTING A SESSION

- `session_start()` must be called before sending the Web browser any output
- If a client's Web browser is configured to accept cookies, the session ID is assigned to a temporary cookie named `PHPSESSID`
- The session ID can be send as a query string or hidden form field to any Web pages that are called as part of the current session

STARTING A SESSION

- `session_id()` - function that can be used to get or set the session id for the current session
- `SID` - constant that can be used to retrieve the current name and session id (string suitable for adding to URLs)

```
<?php
session_start();
...
?>
<p><a href='<?php echo "Occupation.php?PHPSESSID="
                . session_id(); ?>'>Occupation</a></p>
OR
<p><a href='<?php echo "Occupation.php?"
                . SID; ?>'>Occupation</a></p>
```

SESSION VARIABLES

- Session state information (session variables) are stored in the `$_SESSION` autoglobal
- When the `session_start()` is called
 - PHP either initializes a new `$_SESSION` autoglobal, or
 - retrieves any variables for the current session (based on the session ID) into the `$_SESSION` autoglobal

SESSION VARIABLES

```
<?php
session_start();
$_SESSION['firstName'] = "Elena";
$_SESSION['lastName'] = "Vlahu";
$_SESSION['occupation'] = "lecturer";
?>
---
<p><a href='<?php echo "Occupation.php?"
    . SID ?>'>Occupation</a></p>
```

SESSION VARIABLES

- `isset()` function can be used to ensure that a session variable is set

```
<?php
session_start();
if (isset($_SESSION['firstName']) &&
    isset($_SESSION['lastName']) &&
    isset($_SESSION['occupation']))
    echo "<p>" . $_SESSION['firstName'] . " "
        . $_SESSION['lastName'] . " is a "
        . $_SESSION['occupation'] . "</p>";
?>
```

DELETING A SESSION

- `session_destroy()` - function that can be used to delete a session manually

MANAGING STATE INFORMATION

- Some *php.ini* settings
 - session.use_strict_mode = 0
 - session.use_cookies = 1
 - session.use_only_cookies = 0
 - session.name = PHPSESSID
 - session.use_trans_sid = 0 / session.use_trans_sid = 1

MANAGING STATE INFORMATION

- Example – College Internship

